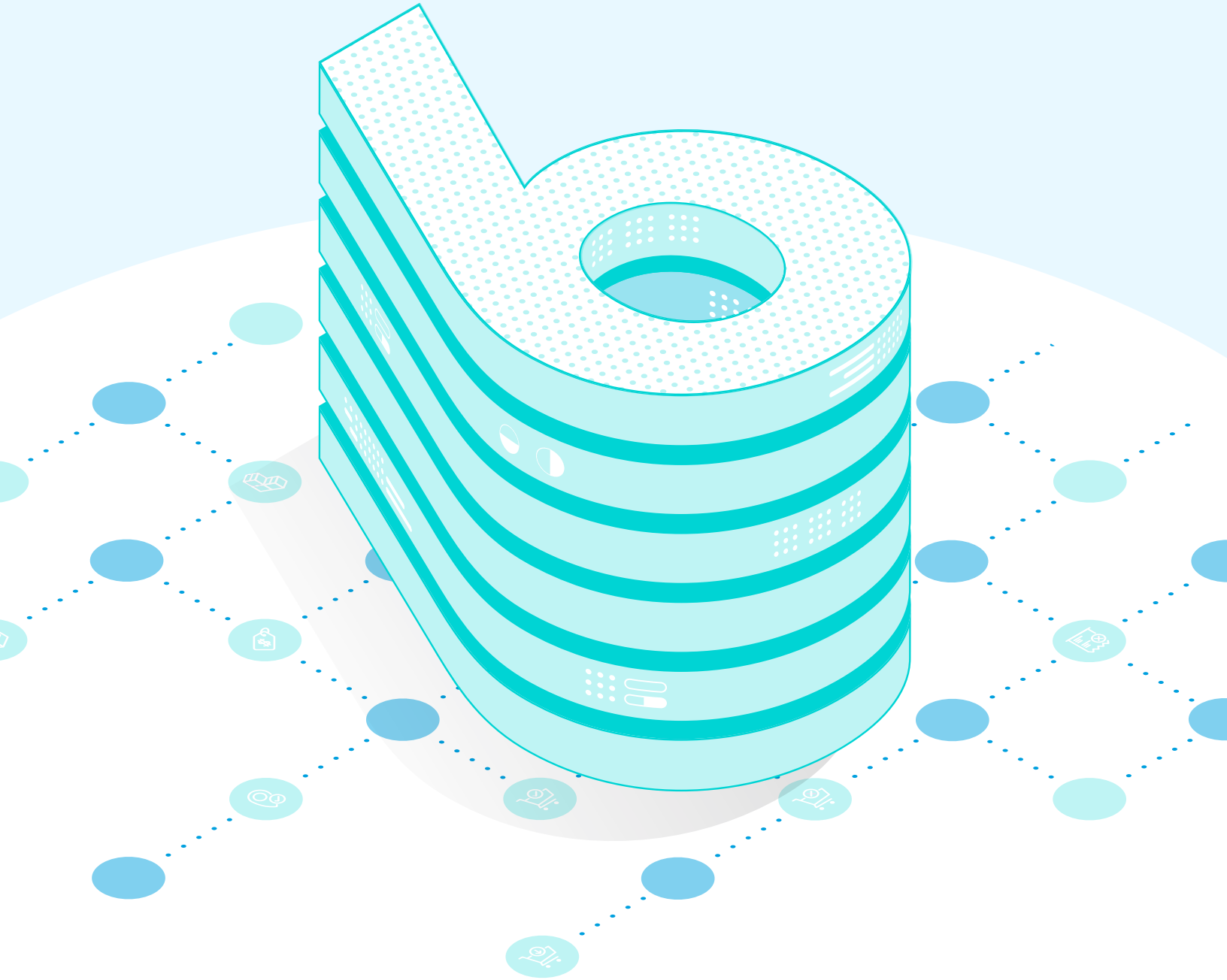
MuleSoft

WHITEPAPER

Top 6 microservices patterns



Contents

What are microservices? 03

Top 6 microservices patterns 05

- 01 Fine-grained SOA 06
- 02 Layered APIs over fine-grained SOA 08
- 03 Message-oriented state management over layered APIs 11
- 04 Event-driven state management over layered APIs 13
- 05 Isolating state in layered APIs 16
- 06 Replicating state in layered APIs (event sourcing) 18

Dive deeper into microservices 20

What are microservices?

Digital transformation has become a race toward agility. The faster an organization can adjust and innovate ahead of customer demands, the more likely they are to become a market leader – while those who trail behind are in a prime position for disruption.

DEFINITION

microservices | mīkrōsərvəs | noun

1. an architectural pattern that breaks down large application structures into smaller, independent services that are not dependent upon a specific coding language.

   — derived from service-oriented architecture (SOA).

2. an application, system, or action that is part of a set of loosely-coupled services.

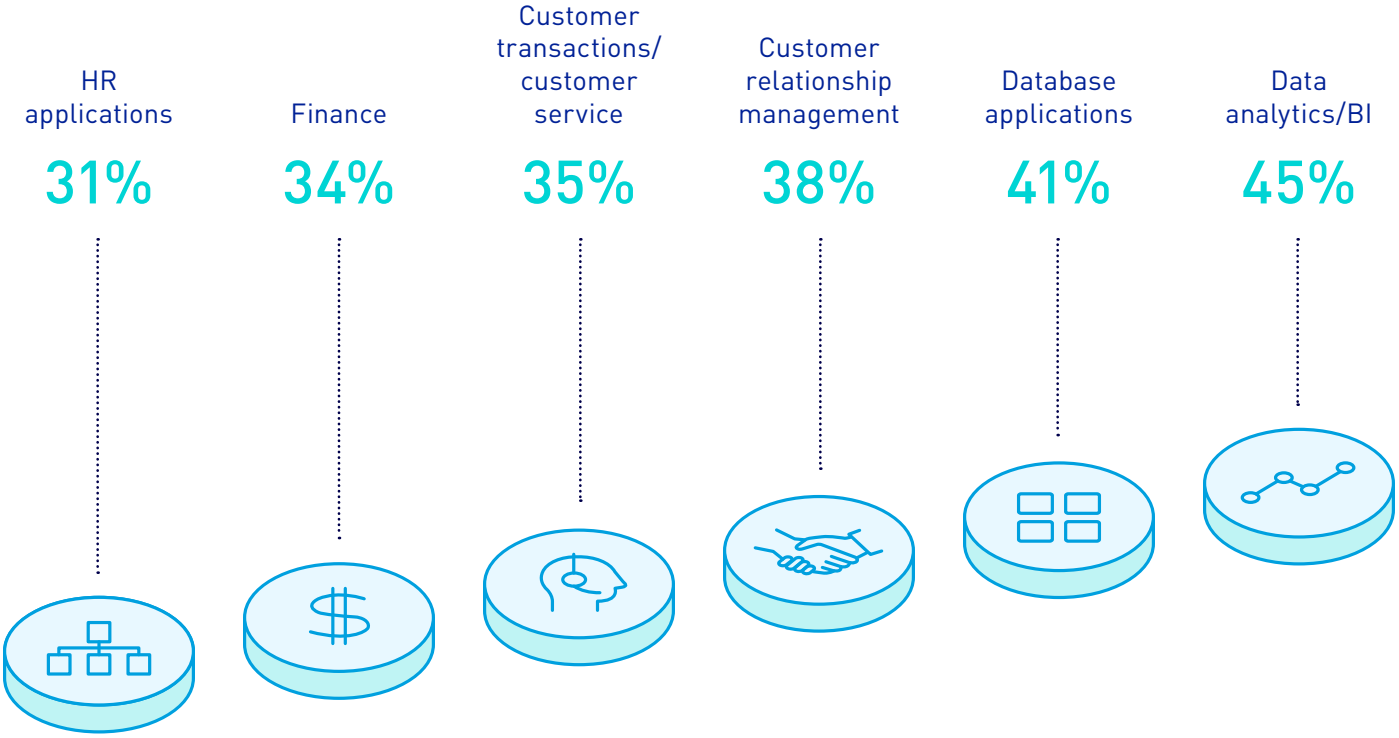
A microservices architecture is the foundation for continuous innovation and agility that enables enterprises to rapidly respond to new business demands. Once considered one of the hottest up-and-coming technologies, microservices have now seen widespread adoption as a way to deconstruct the large, complex infrastructures within organizations across industries.

Traditional applications act as monoliths – meaning that they are single self-contained artifacts – like a bulky concrete slab. A microservice-based application is made up of several building blocks that can be composed together to get new applications and services up and running faster. This type of architecture can encompass thousands of individual components.

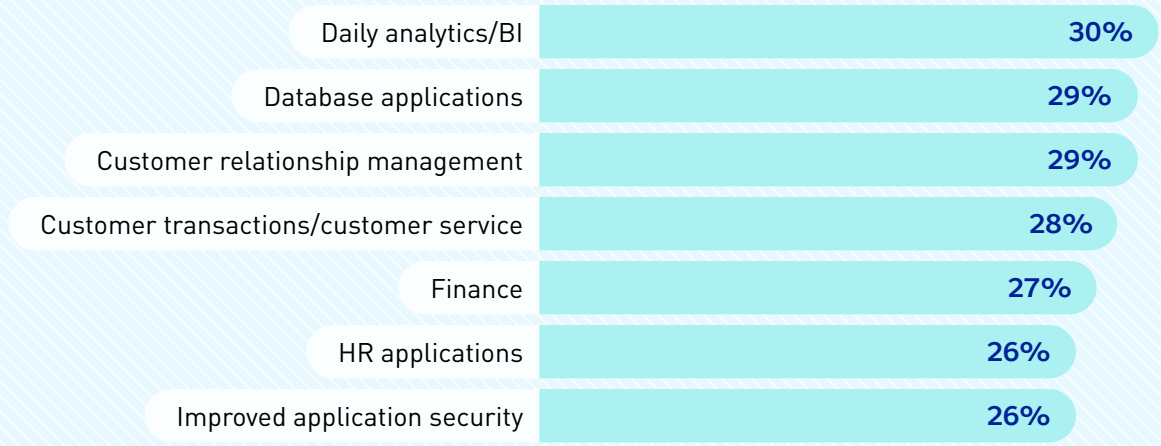
©2022 MULESOFT — MULESOFT.COM

BY THE NUMBERS

Microservices use cases



Benefits of microservices



SOURCE: IBM, MICROSERVICES IN THE ENTERPRISE, 2021: REAL BENEFITS, WORTH THE CHALLENGES

©2021 MULESOFT — MULESOFT.COM

Top 6 microservices patterns

This whitepaper will highlight six of the most common patterns for microservices architectures. No pattern is one-size-fits-all, and each has its own set of benefits and prerequisites. Organizations can choose which pattern is best for their needs or mix and match patterns based on their ecosystem's unique services and structures.

01 Fine-grained SOA

02 Layered APIs over fine-grained SOA

03 Message-oriented state management over layered APIs

04 Event-driven state management over layered APIs

05 Isolating state in layered APIs

06 Replicating state in layered APIs (event sourcing)

©2022 MULESOFT — MULESOFT.COM

01 Fine-grained SOA

Fine-grained SOA is the most common approach to microservices when just getting started. This pattern applies the same principles as SOA but reduces the issues that would typically arise by breaking down the infrastructure into smaller, more granular pieces.

©2022 MULESOFT — MULESOFT.COM

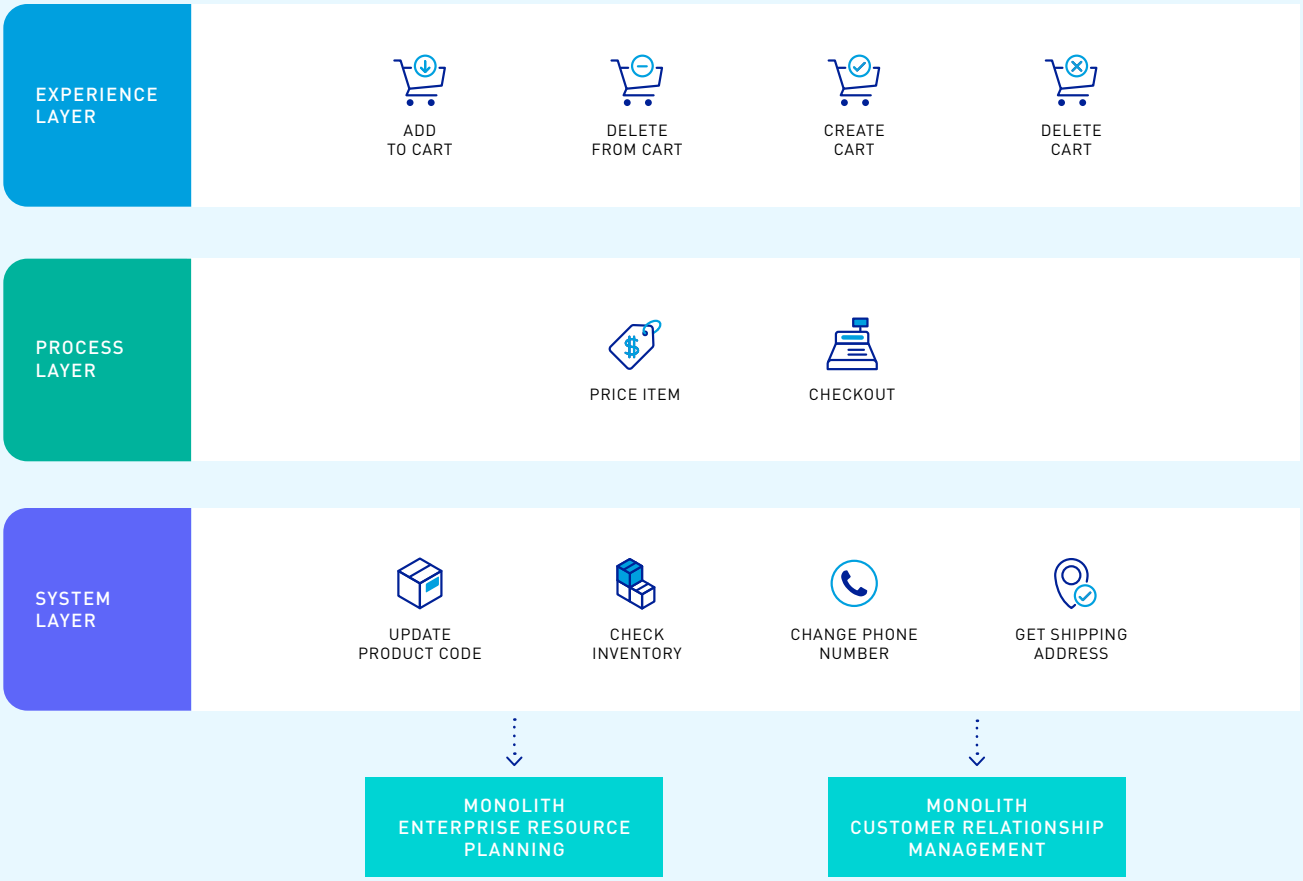
In most cases, this pattern is applied as an extension of service-oriented integration, where each service provides connectivity to external systems. This forms tight dependencies to those external stores, which reduce the speed of change and makes the system's cohesion reflective of the internal state of those applications. Fine-grained SOA is often an entry point into microservices, and enterprises will typically apply more advanced patterns down the road.

KEY CHARACTERISTICS

- Works well at low scale, but not efficient at larger scale.
- Integration-centric with each microservice depending on external systems.
- Focuses on small services.
- Lacks high speed of change due to inefficient interprocess communication.
- Poor data consistency due to lack of state management.
- Similar challenges as SOA.
- Existing management systems can't handle the increased number of services that need to be managed.

©2022 MULESOFT — MULESOFT.COM

02 Layered APIs over fine-grained SOA



The next evolution of the fine-grained SOA pattern is to layer APIs over it – in most cases, these two patterns will coexist. The layered API over the fine-grained SOA approach is closely related to the concept of API-led connectivity. In both, System APIs expose applications, Process APIs orchestrate them, and Experience APIs provide end-user experiences.

Microservices architectures can be difficult to rationalize when there is a lack of structure, making it hard to categorize and visualize the purpose of each microservice. This pattern creates some structure by organizing the microservices into layers grouped by purpose (systems, processes or domain models, and experiences) and enables the architecture to be managed more easily.

©2022 MULESOFT — MULESOFT.COM

Given that significantly more components are managed in this pattern, it is often too complex for the average enterprise to handle alone. This complexity leads to the adoption of an API gateway or service mesh to ensure each component performs properly, integrates into the larger ecosystem, and is secured.

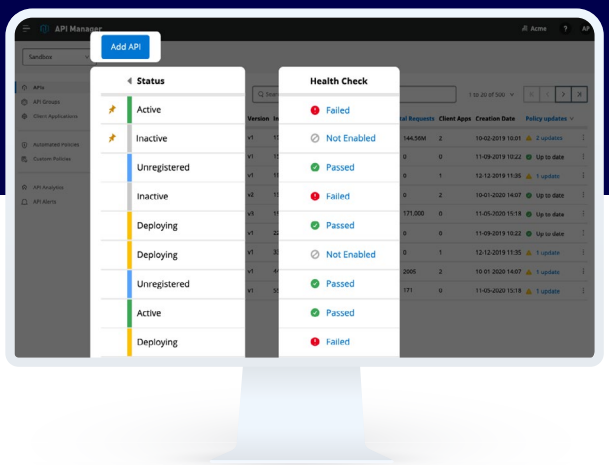
KEY CHARACTERISTICS

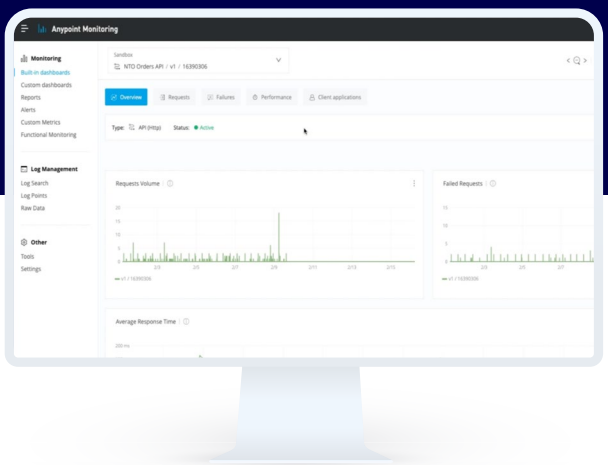
- Works well at low scale, but can be difficult for large-scale applications.
- Increased flexibility while also structuring the architecture to separate concerns.
- Multiple calls are needed through each of the layers.
- Existing management tools no longer work effectively since they can't visualize complex microservices views.
- End-application datastore consistency is improved.
- Additional technologies – such as a service mesh or API gateway – are needed to manage and secure all elements within the architecture.
- Reduced speed of change due to each system microservice depending on external systems.
- Lacks state management capabilities.
- Allows for a high degree of reuse.
- High cohesion due to structured architectural approach.

©2022 MULESOFT — MULESOFT.COM

Extend Anypoint Platform to manage and secure any microservice

Manage and secure all of your microservices and APIs with a single control plane and deploy them anywhere with Anypoint API Manager and the all-new ultrafast Flex Gateway.





FLEX GATEWAY

Built for developers

→ Manage locally with declarative configuration files.

→ Deploy anywhere – with support for cloud instances, container orchestrators, Kubernetes, and bare metal – with one flexible gateway.

Learn more

API MANAGER

Loved by enterprises

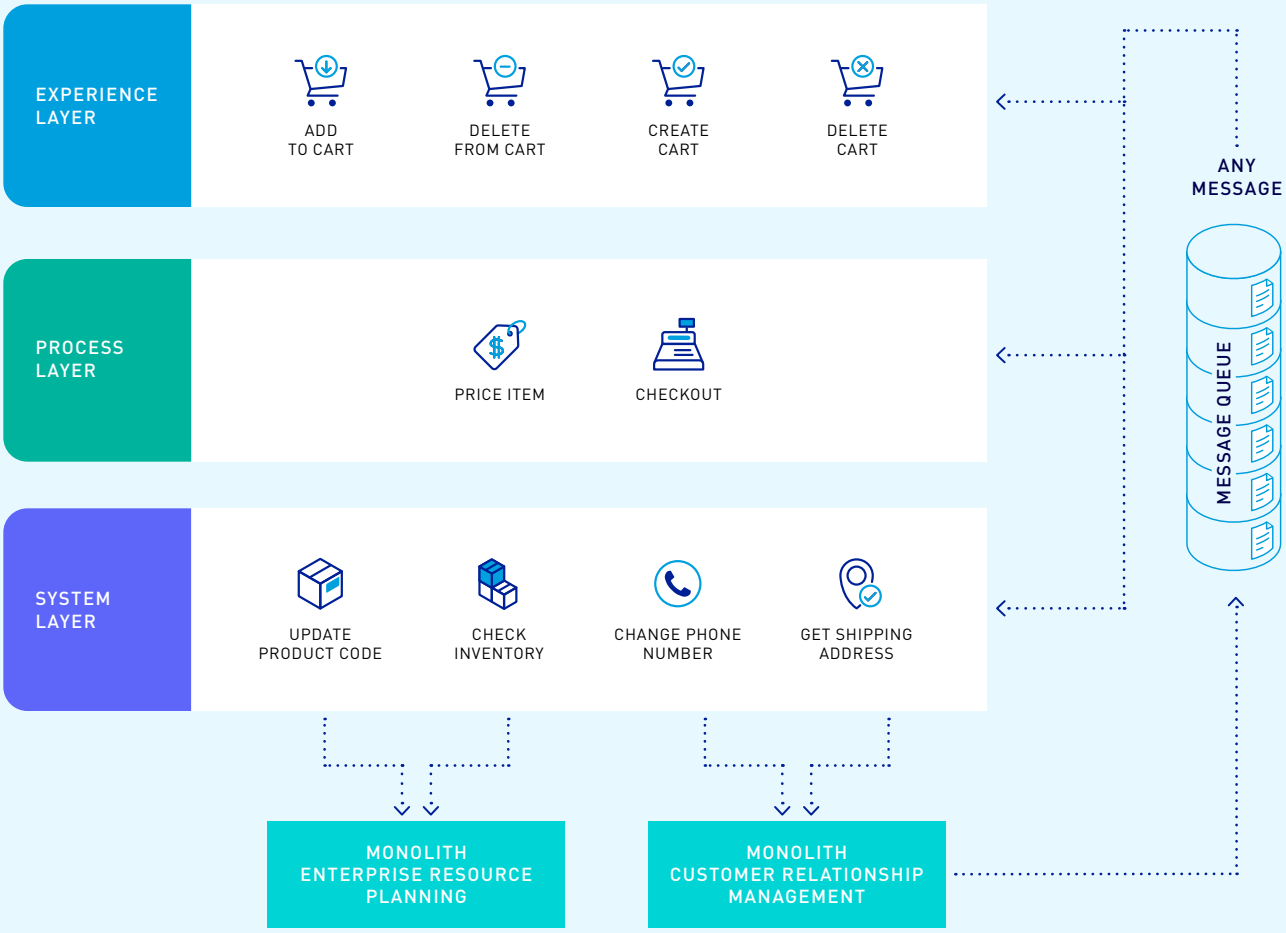
→ Apply pre-built or custom policies to individual or groups of microservices and APIs with guided experience.

→ Centrally manage, secure, and observe all microservices and APIs across the enterprise.

Learn more

©2022 MULESOFT — MULESOFT.COM

03 Message-oriented state management over layered APIs



The prior two patterns lacked state management capabilities, meaning that the microservices implemented using them lacked data integrity. The message-oriented state management over layered API patterns ensures data integrity by replicating the state of key business data between microservices or data stores.

©2022 MULESOFT — MULESOFT.COM

This enables the systems to converge events and provide a consistent external view using a message queue – allowing the state to be asynchronously sent to different locations or queried through other microservices. By decoupling components, the implementation and behavior of each microservice become obscured. Organizations just starting their microservices journey and interested in managing stateful applications frequently use this pattern, and their approach often evolves as it becomes more mature.

KEY CHARACTERISTICS

Increased complexity with state management capabilities.

Implementations can be inconsistent and hard to reuse since there are no standard patterns.

Lack of solutions in the case of data conflicts or rebuilding state due to failures or outages.

Efficient inter-process communication due to asynchronicity.

Data consistency and state management are impacted by the unpredictability of behavior and the potential side effects of a given message.

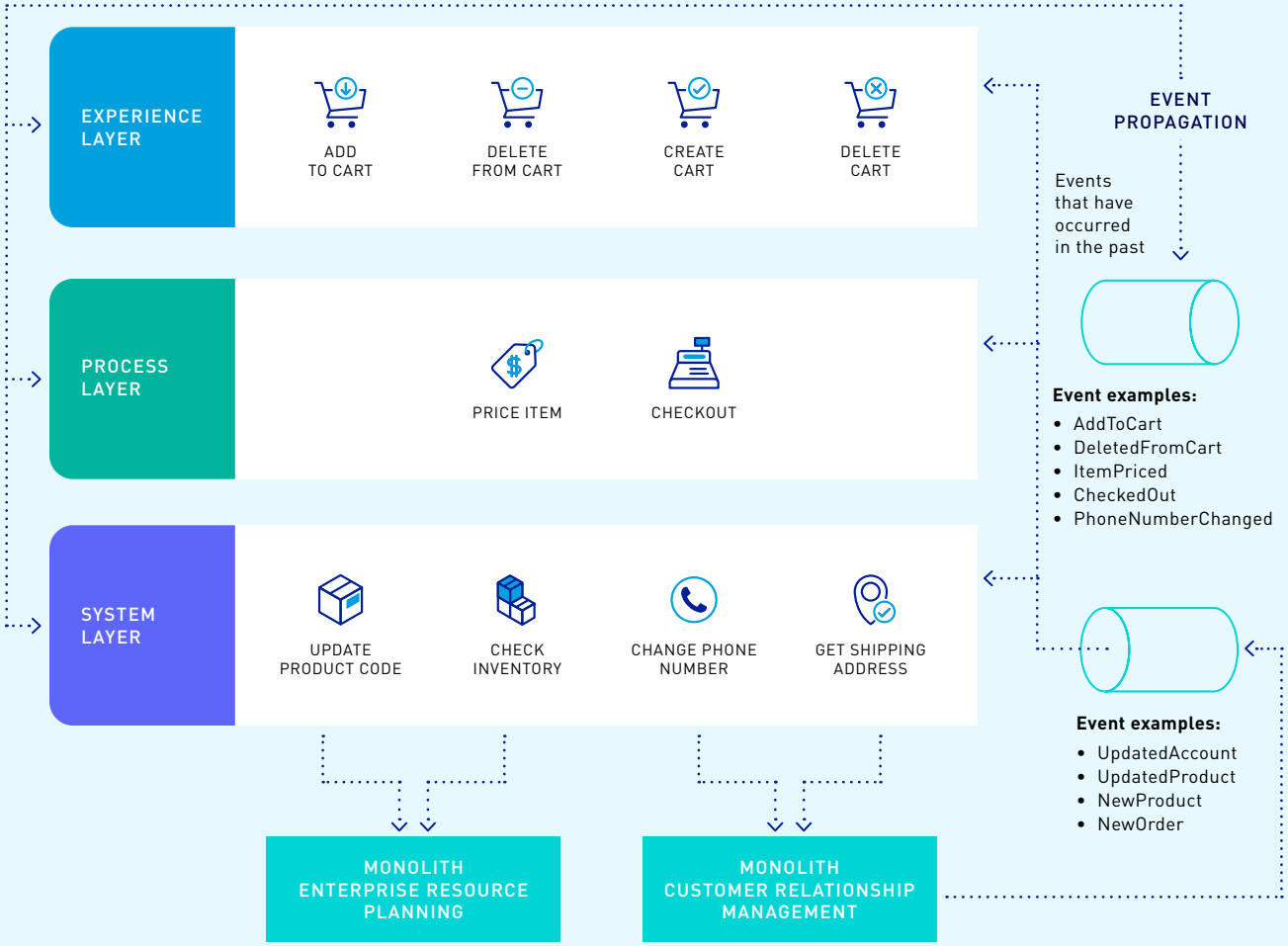
Basic level queueing, but standards around what is passed over the queue are needed.

It works well at a high scale level, but tends to become operationally unpredictable.

There is poor cohesion due to lack of design standards.

©2022 MULESOFT — MULESOFT.COM

04 Event-driven state management over layered APIs



Organizations leverage event-driven patterns for real-time data updates that REST APIs and synchronous communications don't provide; this is critical for use cases like fraud detection, workflow notifications, news feeds, and stock ticks. Event-driven systems use a queue (like message-oriented systems) but have standards enforced for the design and behavior of what is passed over the queue – specifically, an event.

©2022 MULESOFT — MULESOFT.COM

An event is an action associated with a representative state and timestamp. This event allows any service receiving it to reconstruct a materialized view of the state by replaying the events in order.

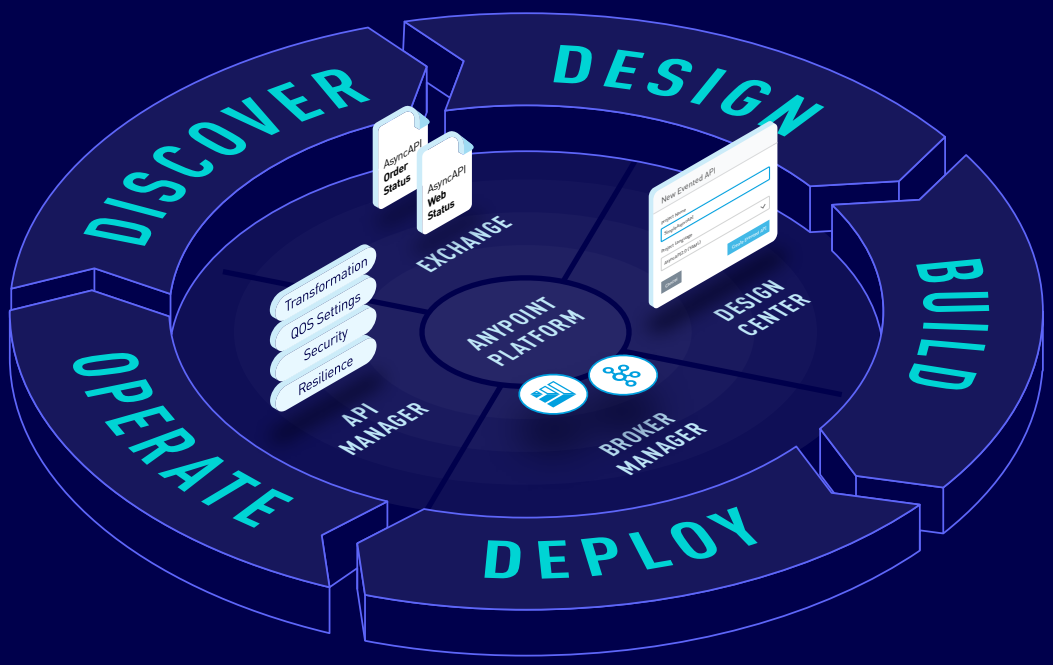
While microservices are often better at scaling and recovering when asynchronous interactions are involved, they are difficult to manage compared to RESTful interactions. To get the best of both worlds, organizations need a solution that offers **full lifecycle management** for microservices – RESTful and event-driven – under a single pane of glass.

KEY CHARACTERISTICS

- Can increase complexity with state management capabilities.
- Implementation can be inconsistent since there are no standard patterns.
- Lack of solutions in the case of data conflicts or rebuilding state due to failures or outages.
- There is efficient inter-process communication due to asynchronicity.
- The flexibility of design is lost in favor of predictable behavior.
- Data consistency and state management are improved through a specific model of consistency.
- Can create complications when events are mixed up with commands.
- Effective scaling model with reasonable operational oversight.
- Strong cohesion when applied consistently, but cohesion tends to drift over time.

©2022 MULESOFT — MULESOFT.COM

Manage the full lifecycle of your event-driven APIs

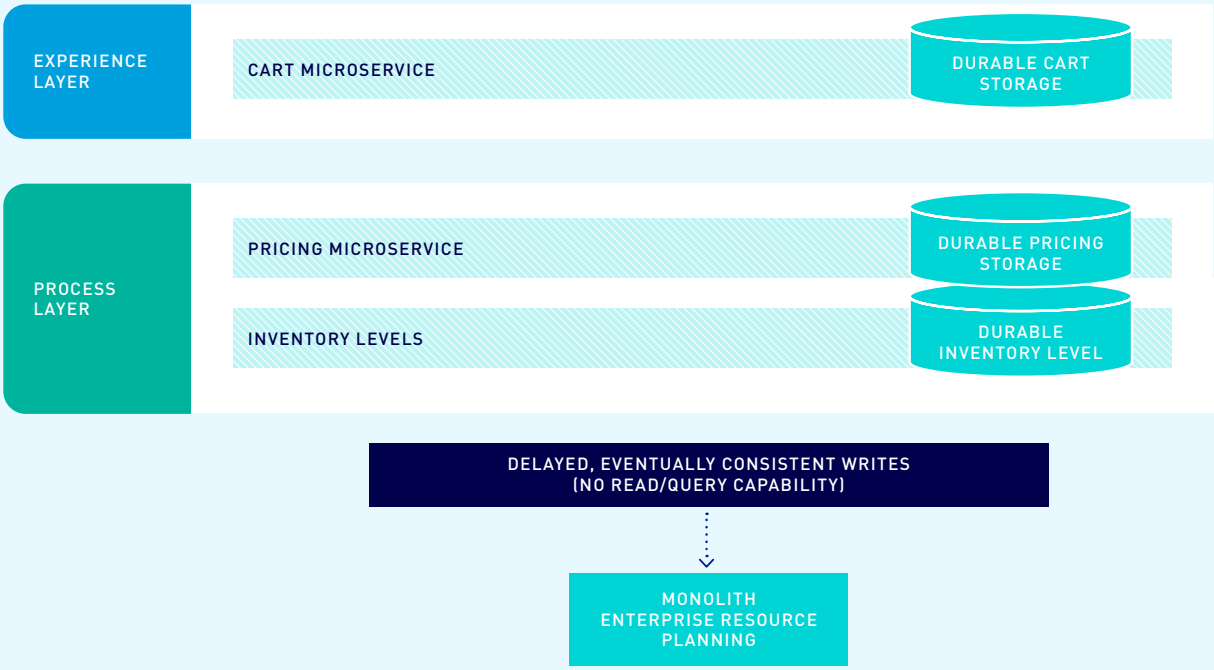


Anypoint Platform allows customers to build and manage the full lifecycle of event-driven and RESTful APIs under a single control plane.

- → Discover and reuse events with AsyncAPI.
- → Establish standards in defining an event-driven API.
- → Interact with event brokers.
- → Manage and secure event-driven APIs and policies.

©2022 MULESOFT — MULESOFT.COM

05 Isolating state in layered APIs



Another alternate approach to event-driven microservices is to add persistence at each individual microservice. This pattern implements consistency at the time of query instead of in the interchange and achieves this by isolating the state of each microservice so each contains its own state.

Each microservice becomes a single source of truth within itself in this pattern, containing an internal data store that constantly reconciles with external stores, whether they are an event log or an enterprise asset. Single source of truth patterns tend to see complex issues (similar to the issues seen with master data management) but this can be simplified when an external store is used.

©2022 MULESOFT — MULESOFT.COM

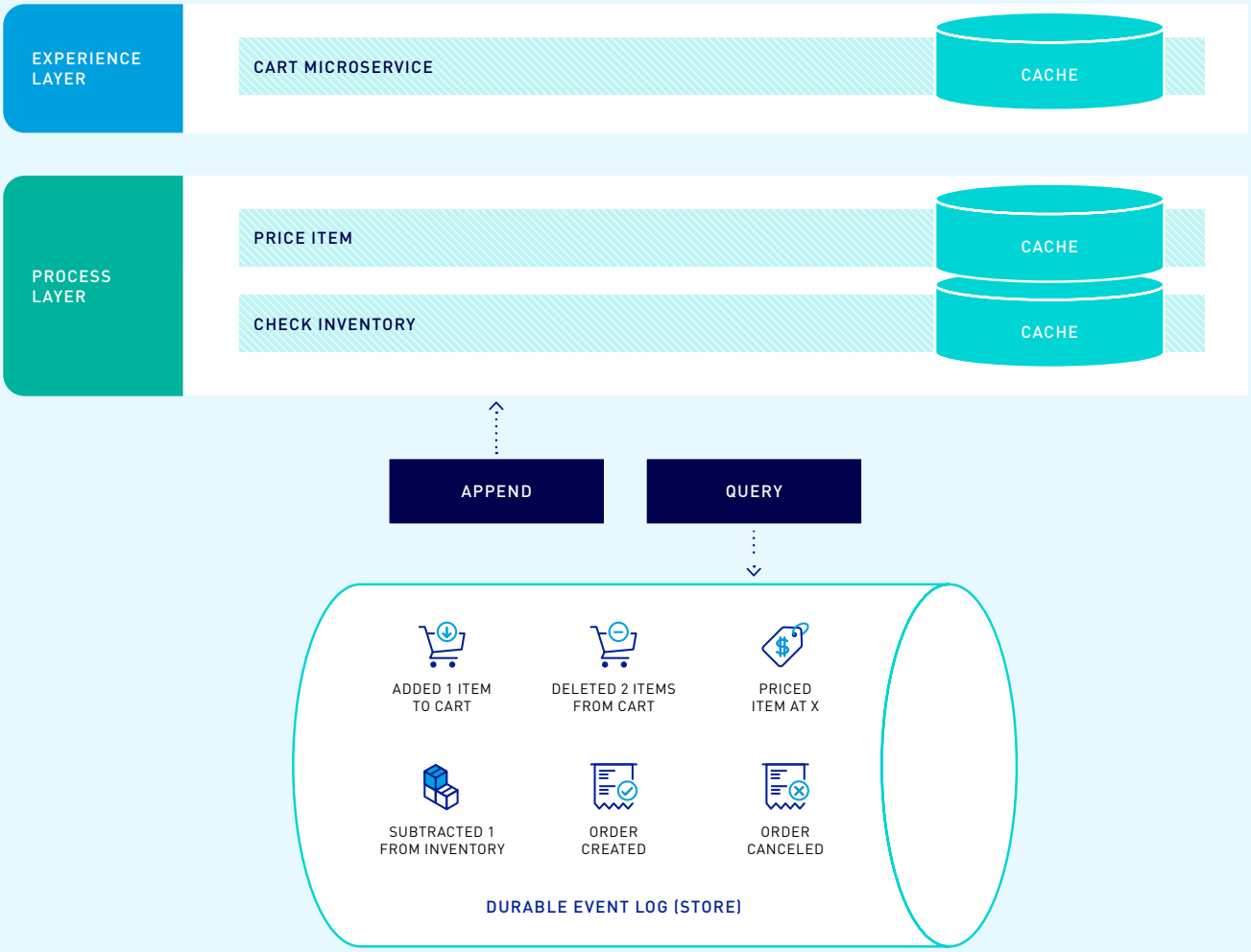
This pattern favors granular microservices. For example, it's difficult to isolate the state of a customer, but it's easy to isolate the state of the customer's email address.

KEY CHARACTERISTICS

- Easy-to-use architecture due to its standardized nature.
- Requires some governance to ensure that data is not copied or has other modes of access.
- Can be difficult to enforce when working with existing assets, such as ERPs.
- No easy fallback position if data entities get out-of-sync.
- Reuse is not a priority.
- There is efficient inter-process communication due to asynchronicity.
- Flexible design increases the speed of change.
- Scalability may pose challenges due to data store scaling.
- Difficult to scalably divide a data model into independent pieces.

©2022 MULESOFT — MULESOFT.COM

06 Replicating state in layered APIs (event sourcing)



A replicating state is essentially the opposite of an isolating state, since consistency is required. With isolated states, microservices become interdependent and failure in a single microservice can cause others to fail. A replicating state provides a single place to store all state mutations where each isolated microservice can rebuild its internal state.

©2022 MULESOFT — MULESOFT.COM

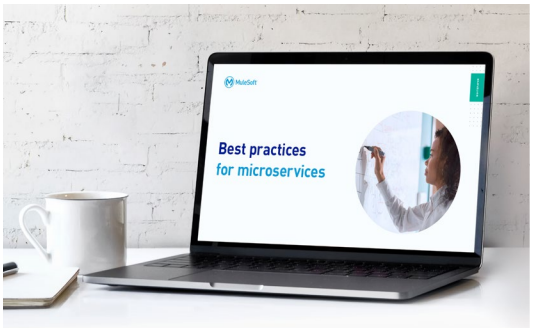
Replicating state requires a deeper understanding of management processes, and behaviors of each microservice to be predictable. By nature, this design is eventually consistent. While this may seem like a problem in traditional transactional design, it is alleviated with insight into the nature of the design. This enables greater freedom and speed of change and ultimately faster time to value.

KEY CHARACTERISTICS

- Creates a cohesive architecture.
- Extremely scalable due to the inherent Command-Query Request separation in the design.
- Often coexists with event sourcing.
- Can be difficult to visualize and understand logical dependencies.
- Efficient inter-process communication due to asynchronicity.
- Flexible design increases the speed of change.
- Data consistency is moderate, but achieves a single source of truth.
- Scalability is effective as each piece can scale independently.

©2022 MULESOFT — MULESOFT.COM

Dive deeper into microservices

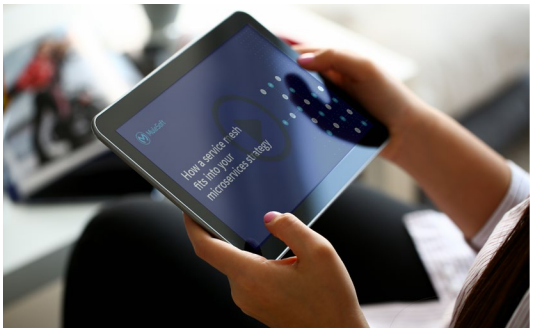


WHITEPAPER

Best practices for microservices

Get the microservices best practice guide with 10 steps for proper implementation.

Download whitepaper



WEBINAR

How a service mesh fits into your microservices strategy

See how to uplevel your microservice strategy with a service mesh.

Watch webinar

WHITEPAPER

Microservices security and digital risk management

Learn how to secure and manage your microservice architecture.

Download whitepaper

©2022 MULESOFT — MULESOFT.COM

MULESOFT, A SALESFORCE COMPANY

MuleSoft, provider of the world's #1 trusted integration and API platform, empowers any company to quickly unlock and integrate their apps and data to create seamless experiences, faster.
For more information, visit: mulesoft.com

MULESOFT IS A REGISTERED TRADEMARK OF MULESOFT, INC., A SALESFORCE COMPANY.
ALL OTHER MARKS ARE THOSE OF RESPECTIVE OWNERS.